AuthService Project

Cursor Initial Codebase Analysis — PDF Summary

Prepared from the analysis provided by Cursor. This document preserves the findings and caveats from that report; suspected issues and unverified runtime behavior are not presented as confirmed runtime failures.

Contents

1. Project purpose and functionality
2. Technologies and frameworks
3. Package structure
4. API endpoints
5. Entities, DTOs, repositories, services and security
6. Authentication flows
7. Dependencies
8. Entry point and database configuration
9. Interview recap
10. Items not inspected

This analysis is based on the **AuthService** Gradle multi-project under `E:\Switch_Prep\LovePreet_Project_ExpenceTracker\AuthService`, with application code in the `app` module. No files were modified.

1. Project purpose and main functionality

AuthService is a **stateless JWT authentication microservice**. From the parent path (`LovePreet_Project_ExpenceTracker`) it is intended as the auth layer of an expense-tracker system. The code itself never mentions expenses; it only handles identity and tokens.

What the code actually does:

Capability	Evidence
User signup	`POST /auth/v1/signup` → BCrypt password, UUID `userId`, persist `UserInfo`
Login	`POST /auth/v1/login` → `AuthenticationManager` + username/password → JWT + refresh token
Refresh access token	`POST /auth/v1/refreshToken` → lookup/expiry check → new JWT
JWT on later requests	`JwtAuthFilter` reads `Authorization: Bearer ...`

Not implemented (no code found): logout, token revocation, password reset, email verification, role assignment at signup, OAuth2 login, Kafka/event publishing, user profile CRUD, expense APIs, tests, Docker.

Signup stores an **empty role set**. `firstName` / `lastName` / `phoneNumber` / `email` on `UserInfoDto` are **not persisted**. Comments `ValidationUtil.validateUserAttributes` and `pushEventToQueue` are unimplemented.

2. Technologies and frameworks

Area	What is used
Language	Java **21** (`sourceCompatibility` / toolchain)
App	**Spring Boot 3.2.2**
Build	Gradle 8.x (`settings.gradle` cites Gradle 8.5 docs), Java plugin + Spring Boot plugin
Web	`spring-boot-starter-web`
Security	`spring-boot-starter-security`, custom JWT filter, `DaoAuthenticationProvider`
Persistence	Spring Data JPA + Hibernate (`ddl-auto=update`)
DB	MySQL (`mysql-connector-j`, `MySQL8Dialect`)
JWT	JJWT **0.12.5** (api + impl + jackson), HS256
Password	BCrypt (`BCryptPasswordEncoder`)
Boilerplate	Lombok 1.18.30
Observability	`spring-boot-starter-actuator` (on classpath; no custom actuator config)

On the classpath but unused in source:

- `spring-boot-starter-oauth2-resource-server` — no `oauth2ResourceServer()`, JWT decoder, or issuer config
- `spring-cloud-starter-bootstrap` — no `bootstrap.yml` / `bootstrap.properties`
- `org.modelmapper:modelmapper` — no `ModelMapper` bean or mapping usage
- Guava — no imports
- JUnit 4 — no `src/test` tree

IDE: Eclipse Buildship (`.project`, `.classpath`). Gradle root is the parent (`connection.project.dir=..`).

3. Package and folder structure

```
AuthService/                          # Gradle root: rootProject.name = 'AuthService'
■■■ settings.gradle                    # include('app'), foojay JDK resolver
■■■ gradle/libs.versions.toml          # guava 32.1.2-jre, junit 4.13.2
■■■ gradlew
■■■ .gitignore                         # .gradle, build
■■■ app/                               # Spring Boot module
    ■■■ build.gradle
    ■■■ .project / .classpath / .settings/
    ■■■ build/                         # compiled output (bootJar, scripts)
    ■■■ src/main/
        ■■■ resources/
        ■   ■■■ application.properties
        ■■■ java/authservice/
            ■■■ App.java
            ■■■ auth/                  # SecurityConfig, JwtAuthFilter, UserConfig
            ■■■ controller/            # AuthController, TokenController
            ■■■ entities/              # UserInfo, UserRole, RefreshToken
            ■■■ model/                 # UserInfoDto
            ■■■ request/               # AuthRequestDTO, RefreshTokenRequestDTO
            ■■■ response/              # JwtResponseDTO
            ■■■ repository/            # UserRepository, RefreshTokenRepository
            ■■■ service/               # JwtService, UserDetailsServiceImpl,
            ■                         # RefreshTokenService, CustomUserDetails
            ■■■ utils/                 # ValidationUtil (empty class)
```

Not present: `src/test`, `src/main/resources` besides `application.properties`, `RoleRepository`, exception handlers, Docker, README, parent `build.gradle`.

Java packages are `authservice.*` **(all lowercase)**. `App.java` also references `authService.*` (camel-case). That mismatch is important for component/JPA scanning (section 8).

4. Controllers and API endpoints

There are **two controllers**, **three POST endpoints**. No class-level `@RequestMapping`. Server port **9898**.

`AuthController` (`@RestController`)

Method	Path	Request body	Success	Failure
POST	`/auth/v1/signup`	`UserInfoDto`	`200` `JwtResponseDTO` (access + refresh)	`400` `"Already Exist"`; `500` `"Exception in User Service"`

Flow: `signupUser` → `createRefreshToken(username)` → `jwtService.GenerateToken(username)`.

`TokenController` (`@Controller`, **not** `@RestController`)

Method	Path	Request body	Return
POST	`/auth/v1/login`	`AuthRequestDTO`	`ResponseEntity` with `JwtResponseDTO` or `500`
POST	`/auth/v1/refreshToken`	`RefreshTokenRequestDTO`	raw `JwtResponseDTO` (no `ResponseEntity`)

Login uses `authenticationManager.authenticate(UsernamePasswordAuthenticationToken)`. If `isAuthenticated()` is false it returns 500 (in practice a failed auth usually **throws** `AuthenticationException` before that branch).

Interview note: `@Controller` without `@ResponseBody` means `refreshToken` returning `JwtResponseDTO` is treated as a **view name**, not JSON. `login` returning `ResponseEntity` still writes the body. Signup is fine because of `@RestController`.

Permit-all (no JWT): `/auth/v1/login`, `/auth/v1/refreshToken`, `/auth/v1/signup`. **Everything else** (including default Actuator paths if enabled) requires authentication.

No GET/PUT/DELETE endpoints and no `@PreAuthorize` / method-security usage despite `@EnableMethodSecurity`.

5. Entities, DTOs, repositories, services, security, config

Entities (JPA)

`UserInfo` → table `users`

- PK `user_id` (`String`, assigned UUID, not generated)
- `username`, `password`
- `@ManyToMany(EAGER)` with `UserRole` via join table `users_roles` (`user_id`, `role_id`)

`UserRole` → table `roles`

- PK `role_id` (`GenerationType.AUTO`)
- `name` (mapped to `SimpleGrantedAuthority` as `name.toUpperCase()`)

There is **no** `RoleRepository` and signup never inserts roles.

`RefreshToken` → table `tokens`

- PK `id` (`IDENTITY`)
- `token` (UUID string), `expiryDate` (`Instant`)
- `@OneToOne` to `UserInfo` with `@JoinColumn(name = "id", referencedColumnName = "user_id")`

That join column named `id` is the same as the token PK. Typical design would be a separate `user_id` FK. `@JsonNaming(SnakeCaseStrategy)` is on the entity; it is not an API DTO.

DTOs / request / response

Class	Package	Fields	Role
`UserInfoDto`	`model`	extends `UserInfo` + `firstName`, `lastName`, `phoneNumber`, `email`; snake_case JSON	signup body
`AuthRequestDTO`	`request`	`username`, `password`	login
`RefreshTokenRequestDTO`	`request`	`token`	refresh
`JwtResponseDTO`	`response`	`accessToken`, `token` (refresh)	auth responses

Extending the **entity** as a DTO is a common interview anti-pattern (leaks persistence, extra unused fields).

Repositories

- `UserRepository extends CrudRepository<UserInfo, String>` — `findByUsername`
- `RefreshTokenRepository extends CrudRepository<RefreshToken, Long>` — `findByToken`

No `JpaRepository`, no custom queries, no uniqueness constraint in code for `username`.

Services

Class	Annotation	Role
`JwtService`	`@Service`	create/parse/validate HS256 JWT; subject = username; **1 minute** TTL; **hardcoded** Base64 secret
`UserDetailsServiceImpl`	`@Component`	`loadUserByUsername`, duplicate check, signup (encode + save)
`RefreshTokenService`	`@Service`	create (TTL **600_000 ms = 10 min**), find, delete-if-expired
`CustomUserDetails`	none	extends `UserInfo`, implements `UserDetails`; copies username/password/roles

`UserDetailsServiceImpl.signupUser` encodes the password **before** the existence check (harmless but slightly odd). Duplicate user → `false`, not an exception.

Security

Class	Role
`SecurityConfig`	CSRF off, CORS off, permit-all auth paths, **STATELESS**, HTTP Basic **and** JWT filter, `DaoAuthenticationProvider`, `AuthenticationManager`
`JwtAuthFilter`	`OncePerRequestFilter`: Bearer token → `extractUsername` → `loadUserByUsername` → `validateToken` → `SecurityContext`
`UserConfig`	`PasswordEncoder` bean = `BCryptPasswordEncoder`

`SecurityConfig` also defines a **second** `UserDetailsService` bean (`new UserDetailsServiceImpl(...)`) while `UserDetailsServiceImpl` is already a `@Component`. Duplicate beans are a likely interview talking point.

JWT filter does **not** skip public URLs; it still runs, then the chain continues. Public endpoints work because they are `permitAll`.

Config / utils

- `UserConfig` — password encoder only
- `application.properties` — datasource, JPA, port, security debug logging
- `ValidationUtil` — **empty**
- No `@ControllerAdvice`, no CORS bean, no JWT secret in properties

6. Authentication and authorization flows

A. Signup (`/auth/v1/signup`) — no Spring Security authentication

1. Path is `permitAll`.
2. Password BCrypt-encoded.
3. `findByUsername`; if found → 400.
4. New `UserInfo(userId=UUID, username, encodedPassword, empty HashSet roles)` saved.
5. Refresh token row created; JWT issued.
6. Returns access + refresh. User is **not** put in `SecurityContext`.

B. Login (`/auth/v1/login`)

1. `permitAll`.
2. `AuthenticationManager` → `DaoAuthenticationProvider` → `UserDetailsServiceImpl.loadUserByUsername` → `CustomUserDetails`.
3. Password compared with BCrypt.
4. On success: new refresh token + JWT (same as signup).
5. Failures typically surface as Spring Security 401, not the `else` 500 branch.

C. Refresh (`/auth/v1/refreshToken`)

1. `permitAll` (refresh JWT **without** a valid access token).
2. `findByToken` → `verifyExpiration` (delete + `RuntimeException` if expired) → `getUserInfo` → new JWT.
3. Same refresh token string is returned (not rotated).
4. Missing token → `RuntimeException("Refresh Token is not in DB..!!")` with no handler.

D. Authenticated request (any other path)

1. `JwtAuthFilter` requires `Authorization: Bearer <jwt>`.
2. Parse subject, load user, check username match + not expired.
3. Set `UsernamePasswordAuthenticationToken` with authorities from roles.
4. `anyRequest().authenticated()`.

Authorization: role names become `GrantedAuthority` strings. **No endpoint uses them.** `@EnableMethodSecurity` is unused. Signup never assigns roles, so authorities are usually empty.

Token lifetimes (from code):

- Access JWT: `1000 * 60 * 1` → **60 seconds**
- Refresh: `Instant.now().plusMillis(600000)` → **10 minutes**

Also enabled: `.httpBasic(Customizer.withDefaults())` plus JWT, on a stateless API. Basic Auth is extra surface, not used by the three JSON endpoints.

7. Important `build.gradle` dependencies

Plugins: `application`, `java`, Spring Boot `3.2.2`, dependency-management `1.0.15.RELEASE`.

Main class: `authservice.App`.

Dependency	Why it matters
`spring-boot-starter-web`	REST
`spring-boot-starter-security`	filter chain, `AuthenticationManager`
`spring-boot-starter-data-jpa`	entities/repos
`com.mysql:mysql-connector-j` (runtime)	MySQL
`jjwt-api/impl/jackson:0.12.5`	JWT
`lombok`	`@Data`, `@Builder`, etc.
`spring-boot-starter-actuator`	health/metrics (secured by default config)
`spring-boot-starter-oauth2-resource-server`	**declared, unused**
`spring-cloud-starter-bootstrap:4.1.1`	**declared, unused**
`modelmapper:3.2.0`	**declared, unused**
`libs.guava`	catalog Guava **32.1.2-jre**, unused
`libs.junit`	JUnit **4.13.2**, no tests

`io.spring.dependency-management` 1.0.15 is old relative to Boot 3.2.2 (interview trivia). No Spring Cloud BOM; bootstrap is a pinned version.

8. Entry point and database configuration

Entry point

`authservice.App`:

```
@SpringBootApplication
@EnableJpaRepositories(basePackages = {"authService.repository"})
@ComponentScan(basePackages = {"authService.controller", "authService.auth", "authService.service"})
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

BootJar `Start-Class: authservice.App`, Spring Boot 3.2.2, JDK 21.

Scan issue: real packages are `authservice.repository` etc. Explicit scans use `authService`. On a case-sensitive filesystem that can miss repositories/controllers. `@SpringBootApplication` still scans `authservice` (the class

package). `@EnableJpaRepositories` with the wrong base package can **override** Boot's default repo scan and fail to register `UserRepository` / `RefreshTokenRepository`. Worth verifying at runtime on Linux vs Windows.

Commented-out `SecurityFilterChain` bean in App is dead code; the live chain is in `SecurityConfig`.

Database (`application.properties`)

Property	Value
Driver	`com.mysql.cj.jdbc.Driver`
URL	`jdbc:mysql://${MYSQL_HOST:localhost}:${MYSQL_PORT:3306}/${MYSQL_DB:authservice}?useSSL=false&useUnicode=yes&characterEncoding=UTF-8&allowPublicKeyRetrieval=true`
User	`${MYSQL_USER:root}`
Password	`${MYSQL_PASSWORD:...}` with a **hardcoded default** in source
`ddl-auto`	`update` (schema created/altered at startup; no Flyway/Liquibase)
Dialect	`MySQL8Dialect`
`show-sql`	true
Port	**9898**

Expected tables from mappings: `users`, `roles`, `users_roles`, `tokens`. No SQL migrations in the repo.

Interview-oriented recap

This is a **classic JWT + refresh-token Spring Security** setup:

`SecurityFilterChain` → `JwtAuthFilter` before `UsernamePasswordAuthenticationFilter` → `DaoAuthenticationProvider` + `UserDetailsService` + BCrypt → JJWT access tokens + DB-backed refresh tokens → stateless sessions.

Gaps to discuss if asked "what would you change?" (observed in code, not speculation):

- Hardcoded JWT secret and default DB password
- 1-minute access token; refresh not rotated
- `TokenController` is `@Controller`; refresh may not return JSON
- Package name mismatch in `@EnableJpaRepositories` / `@ComponentScan`
- Duplicate `UserDetailsService` beans
- DTO extends entity; extra profile fields never saved
- Empty roles; method security unused
- `RefreshToken` `@JoinColumn(name = "id")` vs token PK
- Unused OAuth2 resource-server, ModelMapper, Guava, Cloud bootstrap
- No tests, no global exception handling, CSRF/CORS fully off

Could not inspect

- Full parent directory listing (shell sandbox unavailable); `settings.gradle`, `gradle/libs.versions.toml`, `gradlew`, `.gitignore` **were** read
- Gradle wrapper JAR / `gradle-wrapper.properties`
- Runtime behavior (app was not started; MySQL not verified)
- Whether `authService` vs `authservice` works on this Windows machine
- Compiled classes beyond `build/resolvedMainClassName` and boot JAR manifest
- Any sibling services (expense API, gateway) outside this repo — not in the scanned tree

If you want a follow-up, a good next step is a **whiteboard-style sequence diagram** of login vs JWT filter, or a **defect list prioritized for a code review**, still without changing files.